

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение
высшего образования «Санкт-Петербургский политехнический университет
Петра Великого»

Институт компьютерных наук и кибербезопасности

Высшая школа технологий искусственного интеллекта

Направление: 02.03.01 Математика и компьютерные науки

Функциональное программирование

**Отчет о выполнении лабораторных
работ 1-5**

Студент,
группы 5130201/40003

_____ Лебедев К. О.

Преподаватель,
к.т.н.

_____ Моторин Д.Е.

«_____» _____ 20__ г.

Санкт-Петербург, 2026

Содержание

1	Введение	4
2	Временная диаграмма выполнения заданий	4
2.1	Пояснения к диаграмме	4
2.2	Временная диаграмма	4
3	Структура и содержание репозитория	5
3.1	Общая структура	5
3.2	Содержание разделов	5
4	Лабораторная работа №1	5
4.1	Коммиты и выполняемая работа	5
4.1.1	Коммит 1. Инициализация репозитория (04.03.2026, 16:41)	5
4.1.2	Коммит 2. Добавление myHead и myTail (04.03.2026, 17:05)	5
4.1.3	Коммит 3. Стандартные функции и функции высшего порядка (06.03.2026, 03:59)	6
4.1.4	Коммит 4. myZipSave и myUnzipSave (12.03.2026, 21:19) .	6
4.1.5	Коммит 5. Свёртки и обработка Maybe (13.03.2026, 04:46)	6
4.1.6	Коммит 6. myMap для MyList и myUnfoldr (20.03.2026, 01:23)	6
4.1.7	Коммит 7. Типы для выпекания тортов (20.03.2026, 02:28)	6
4.1.8	Коммит 8. Доработка контроля ингредиентов (26.03.2026, 15:02)	7
4.1.9	Коммит 9. Примеры выпекания (26.03.2026, 16:31)	7
4.1.10	Коммит 10. Реорганизация каталогов (26.03.2026, 17:05) .	7
4.2	Вопросы защиты и их решения	7
4.2.1	Вопрос 1 (задание дано 20.03.2026)	7
4.2.2	Вопрос 2 (задание дано 27.03.2026)	8
5	Лабораторная работа №2	8
5.1	Часть 1. Тип MyEvolution	8
5.1.1	Коммит 1. Проект MyEvolProject и тип MyEvolution (09.04.2026, 19:02)	8
5.2	Часть 2. Типы MyMaybe, MyEither, MyTree	9
5.2.1	Коммит 2. MyTree: Functor (10.04.2026, 00:04)	9
5.2.2	Коммит 3. MyTree: Foldable (15.04.2026, 02:42)	9
5.2.3	Коммит 4. MyEither (16.04.2026, 03:05)	9
5.2.4	Коммит 5. MyTree: Applicative (15.04.2026, 04:13)	9
5.2.5	Коммит 6. MyMaybe (15.04.2026, 05:24)	9
5.3	Часть 3. Парсеры	9
5.3.1	Коммит 7. Собственный парсер MyParser (23.04.2026, 15:51)	9
5.3.2	Коммит 8. ParsecParser (23.04.2026, 20:03)	10

5.3.3	Коммит 9. AttoparsecParser (23.04.2026, 21:19)	10
5.3.4	Коммит 10. Комментарии-промпты (24.04.2026, 11:40)	10
5.4	Вопросы защиты и их решения	10
5.4.1	Вопрос 1 (задание дано 24.04.2026)	10
5.4.2	Вопрос 2 (задание дано 08.05.2026)	10
6	Лабораторная работа №3	11
6.1	Часть 1. Глитч-эффекты	11
6.1.1	Коммит 1. Создание проектов (06.05.2026, 17:36)	11
6.1.2	Коммит 2. Функции глитч-эффектов (07.05.2026, 13:17)	11
6.2	Часть 2. Лабиринт на монаде RWS	11
6.2.1	Коммит 3. Типы данных лабиринта (07.05.2026, 18:50)	11
6.2.2	Коммит 4. Поиск пути (08.05.2026, 00:24)	12
6.2.3	Коммит 5. Тестирование и очистка (08.05.2026, 02:33)	12
6.3	Вопросы защиты и их решения	12
6.3.1	Вопрос 1 (задание дано 08.05.2026)	12
7	Лабораторная работа №4	12
7.1	Коммиты и выполняемая работа	12
7.1.1	Коммит 1. Трансформер MyStateT (14.05.2026, 22:35)	12
7.1.2	Коммит 2. Загрузка лабиринта (14.05.2026, 23:39)	13
7.1.3	Коммит 3. Игровая логика (15.05.2026, 01:32)	13
7.1.4	Коммит 4. Тестовый лабиринт (15.05.2026, 01:33)	13
7.1.5	Коммит 5. Комментарии к runGame (15.05.2026, 17:56)	13
7.2	Вопросы защиты и ответы на них	13
7.2.1	Вопрос 1	13
7.2.2	Вопрос 2	14
7.2.3	Вопрос 3	14
7.2.4	Вопрос 4	14
8	Лабораторная работа №5	14
8.1	Коммиты и выполняемая работа	14
8.1.1	Коммит 1. Создание проекта (21.05.2026, 22:50)	14
8.1.2	Коммит 2. Тестируемые функции (21.05.2026, 23:27)	14
8.1.3	Коммит 3. Генераторы и свойства (22.05.2026, 01:24)	15
8.1.4	Коммит 4. verbose и withMaxSuccess (22.05.2026, 01:29)	15
A	Приложение A. Листинги кода	16

1 Введение

В данном отчёте описывается процесс выполнения лабораторных работ 1–5 по дисциплине «Функциональное программирование». Все работы выполнены на языке Haskell. Для каждой работы приводится перечень коммитов с описанием внесённых изменений, рассмотренные при защите вопросы и их решения. Полные листинги исходного кода вынесены в Приложение А.

Исходный код размещён в репозитории на GitHub с именем `hask_pr01`. Каждая работа находится в отдельном каталоге с именем `practice_` и номером работы (например, `practice_01`).

2 Временная диаграмма выполнения заданий

2.1 Пояснения к диаграмме

В таблице 1 приведены этапы выполнения лабораторных работ с указанием диапазона дат активной работы (по датам коммитов). Работа над заданиями велась последовательно с марта по май 2026 года.

2.2 Временная диаграмма

Таблица 1: Этапы выполнения лабораторных работ

№	Лабораторная работа	Период выполнения
1	Стандартные функции, функции высшего порядка, свёртки, типы для выпекания тортов	04.03 – 27.03
2	Классы типов (<code>Show</code> , <code>Read</code> , <code>Eq</code> , <code>Ord</code> , <code>Enum</code> , <code>Bounded</code>); собственные <code>MyMaybe</code> , <code>MyEither</code> , <code>MyTree</code> ; парсеры	09.04 – 24.04
3	Глитч-эффекты на изображениях; поиск пути в лабиринте на монаде <code>RWS</code>	06.05 – 08.05
4	Собственный трансформер монад <code>MyStateT</code> ; текстовая игра «лабиринт» на стеке трансформеров	14.05 – 15.05
5	Тестирование функций с помощью <code>QuickCheck</code>	21.05 – 22.05

3 Структура и содержание репозитория

3.1 Общая структура

Репозиторий `hask_pr01` организован по каталогам с именем `practice_` и номером работы. В каждом каталоге содержатся только файлы и/или `stack`-проекты, относящиеся к соответствующему заданию.

3.2 Содержание разделов

1. `practice_01` — файлы `pr01_1.hs` и `pr01_2.hs` первой работы.
2. `practice_02/part1/MyEvolProject` — проект первой части второй работы (тип `MyEvolution`).
3. `practice_02/part2/myParser` — `stack`-проект второй и третьей частей второй работы (модули `MyTypes` и `MyParsers`).
4. `practice_03/myGlithes` — первая часть третьей работы (глитч-эффекты).
5. `practice_03/myMaze` — вторая часть третьей работы (`RWS`-решатель лабиринта).
6. `practice_04/myMazeGame` — четвёртая работа (игра на трансформерах монад).
7. `practice_05/myFunctionsAndTests` — пятая работа (`QuickCheck`-тесты).

4 Лабораторная работа №1

В первой работе реализованы стандартные функции работы со списками, функции высшего порядка, свёртки, а также система типов для процесса выпекания тортов с контролем ингредиентов.

4.1 Коммиты и выполняемая работа

4.1.1 Коммит 1. Инициализация репозитория (04.03.2026, 16:41)

Создан репозиторий на GitHub. Добавлены первоначальные файлы первой части задания.

4.1.2 Коммит 2. Добавление `myHead` и `myTail` (04.03.2026, 17:05)

Реализованы первые функции: `myHead` и `myTail` — отделение головы и хвоста списка.

4.1.3 Коммит 3. Стандартные функции и функции высшего порядка (06.03.2026, 03:59)

Реализованы и доработаны функции первой части (`pr01_1.hs`):

- a) `myHead`, `myTail` — отделение головы и хвоста;
- b) `myZip`, `myZip3` — попарное и потройное объединение;
- c) `myUnzip` — разделение списка пар на пару списков;
- d) `myFST`, `mySND`, `myTHRD` — проекции кортежа;
- e) функции высшего порядка: `myMap`, `myZipWith`, `myZipWith3`, `myAll`, `myAny`, `myComposition`.

4.1.4 Коммит 4. `myZipSave` и `myUnzipSave` (12.03.2026, 21:19)

Во второй части (`pr01_2.hs`) реализованы:

- a) `myZipSave :: [a] -> [b] -> ([a, b], [a], [b])` — объединение списков с сохранением остатка более длинного;
- b) `myUnzipSave :: ([a, b], [a], [b]) -> ([a], [b])` — обратная операция с восстановлением остатка.

4.1.5 Коммит 5. Свёртки и обработка `Maybe` (13.03.2026, 04:46)

Реализованы функции на основе свёрток: `myFoldr`, `myFoldl`, `myFoldl1`, `myFoldr1`, `myReverse` (через свёртку), `myTakeWhile` и `mySpan` (через `myFoldr`), а также `myMaybe` — обработка возможно отсутствующего значения.

4.1.6 Коммит 6. `myMap` для `MyList` и `myUnfoldr` (20.03.2026, 01:23)

Определён рекурсивный тип `MyList a = MyEmpty | MyCons a (MyList a)`; для него реализована `myMap`. Добавлена развёртка `myUnfoldr :: (b -> Maybe (a, b)) -> b -> [a]`.

4.1.7 Коммит 7. Типы для выпекания тортов (20.03.2026, 02:28)

Определены типы `Ingredient`, `FillingMix`, `Dough`, `CakeDoughType`, `Cake`, `Action`. Каждый конструктор ингредиента хранит количество (`Int`), что позволяет контролировать объём расхода.

4.1.8 Коммит 8. Доработка контроля ингредиентов (26.03.2026, 15:02)

Уточнены конструкторы ингредиентов (добавлен параметр `Int`); функция `makeCakeMix` исправлена. Функция `cakeDough` откорректирована, вспомогательная функция проверки количества удалена за счёт переноса проверок в охранные выражения.

4.1.9 Коммит 9. Примеры выпекания (26.03.2026, 16:31)

Реализованы `chocolateCakeDough`, `vanillaCakeDough`, `carrotCakeDough` и `chocolateCake`, `vanillaCake`, `carrotCake`. Добавлены примеры удачного приготовления трёх видов тортов и примеры ошибочных ситуаций: недостаток ингредиентов, неверные ингредиенты, несоответствие смеси и теста.

4.1.10 Коммит 10. Реорганизация каталогов (26.03.2026, 17:05)

Файлы перенесены в каталоги `practice_01` и `practice_02`.

4.2 Вопросы защиты и их решения

4.2.1 Вопрос 1 (задание дано 20.03.2026)

Реализовать функцию типа `Maybe (a -> a) -> [Maybe a] -> [Maybe a]`.

```
1 func :: Maybe (a -> a) -> [Maybe a] -> [Maybe a]
2 func Nothing _ = []
3 func _ [] = []
4 func (Just f) (xs) = map g xs where
5     g (Just x) = Just (f x)
6     g Nothing = Nothing
```

Листинг 1: `func` — применение `Maybe`-функции к списку `Maybe`-значений

4.2.2 Вопрос 2 (задание дано 27.03.2026)

Реализовать функцию типа `Either String (a -> b -> c) -> Either String [(a,b)] -> Either String [c]`.

```
1 func0 :: Either String (a -> b -> c) -> Either String [(a,b)] -> Either String
  ↳ [c]
2 func0 (Left a) (Left b) = Left (a ++ b)
3 func0 (Left a) _ = Left a
4 func0 _ (Left b) = Left b
5 func0 (Right f) (Right xs) = Right (helper f xs) where
6   helper f ((a,b): xxs) = (f a b) : helper f xxs
7   helper f [] = []
```

Листинг 2: `func0` — применение `Either`-функции к списку пар `Either`

5 Лабораторная работа №2

Вторая работа состоит из трёх частей: реализация классов типов для перечислимого типа `MyEvolution`; собственные типы `MyMaybe`, `MyEither`, `MyTree` как представители стандартных классов типов; парсеры выражений (собственный и на основе библиотек `Parsec` и `Attoparsec`).

5.1 Часть 1. Тип `MyEvolution`

5.1.1 Коммит 1. Проект `MyEvolProject` и тип `MyEvolution` (09.04.2026, 19:02)

Создан проект `MyEvolProject`. Определён тип-сумма `MyEvolution` с десятью конструкторами — этапами эволюции (от `LUCA` до `Humans`). Вручную реализованы представители классов:

- a) `Show` — каждый конструктор преобразуется в строку (например, `LUCA` → «Last Universal Common Ancestor»);
- b) `Read` — обратное преобразование строки в конструктор;
- c) `Eq` — равенство одинаковых конструкторов;
- d) `Enum` — `fromEnum/toEnum` для порядковых номеров;
- e) `Ord` — сравнение через `fromEnum`;
- f) `Bounded` — `minBound = LUCA`, `maxBound = Humans`.

Дополнительно определён тип `MyEvolution'` с теми же конструкторами, представители классов получены автоматически через `deriving`. В `Main.hs` выводится отсортированный список всех значений типа.

5.2 Часть 2. Типы MyMaybe, MyEither, MyTree

5.2.1 Коммит 2. MyTree: Functor (10.04.2026, 00:04)

Определён тип `MyTree a = Leaf a | Node a (MyTree a) (MyTree a)`, хранящий значения и в листьях, и в узлах. Реализован представитель `Functor`.

5.2.2 Коммит 3. MyTree: Foldable (15.04.2026, 02:42)

Реализован представитель `Foldable` для `MyTree` (`foldr`, `foldMap`, `length`). Добавлены примеры вызовов в комментариях.

5.2.3 Коммит 4. MyEither (16.04.2026, 03:05)

Определён тип `MyEither e a = MyLeft e | MyRight a`. Реализованы представители `Functor`, `Foldable`, `Applicative` и `Semigroup`. Добавлены примеры вызовов всех реализованных функций (`<>`, `stimes`, `sconcat`, `(>*)`, `(<*)`, `liftA2`, `(<$)`, `fold` и другие).

5.2.4 Коммит 5. MyTree: Applicative (15.04.2026, 04:13)

Реализован представитель `Applicative` для `MyTree` (`pure`, `<*>`) и примеры вызовов `<*>`, `liftA2`, `*>`, `<*`.

5.2.5 Коммит 6. MyMaybe (15.04.2026, 05:24)

Определён тип `MyMaybe a = MyJust a | MyNothing`. Реализованы представители `Functor`, `Applicative`, `Semigroup`, `Monoid` (с `empty = MyNothing`) и `Foldable`. В комментариях приведены примеры вызовов `fmap`, `<$`, `pure`, `<*>`, `liftA2`, `<>`, `sconcat`, `stimes`, `mappend`, `mconcat`, `foldr`, `foldMap`, `fold`.

5.3 Часть 3. Парсеры

5.3.1 Коммит 7. Собственный парсер MyParser (23.04.2026, 15:51)

Реализован модуль `MyParsers.MyParser`. Определён тип `newtype Parser tok a = Parser { runParser :: [tok] -> MyMaybe ([tok], a) }`, использующий собственный тип `MyMaybe` вместо стандартного `Maybe`. Реализованы представители `Functor`, `Applicative`, `Alternative`, базовые парсеры (`satisfy`, `char`, `lower`, `digit`) и комбинированные (`multiplication`, `lowers`, `digits`, `finalMult`, `finalPlus`, `plusOrMult`). В `Main.hs` добавлен квалифицированный импорт всех трёх парсеров.

5.3.2 Коммит 8. ParsecParser (23.04.2026, 20:03)

Реализован модуль `MyParsers.ParsecParser` на основе библиотеки `Parsec`: парсеры `digitP`, `multiplicationP`, `digitsP`, `finalMultP`, `finalPlusP`, `plusOrMultPars` (с использованием `try` для `backtracking`). Функция `runParsecParser` запускает парсер и возвращает результат в виде `MyMaybe`.

5.3.3 Коммит 9. AttoparsecParser (23.04.2026, 21:19)

Реализован модуль `MyParsers.AttoparsecParser` на основе библиотеки `Attoparsec`: аналогичные парсеры `digitA`, `multiplicationA`, `digitsA`, `finalMultA`, `finalPlusA`, `plusOrMultAttoparsec`. Функция `runParser` преобразует `String` в `Text` и возвращает результат в виде `MyMaybe`. Добавлен вывод результата для `Attoparsec`.

5.3.4 Коммит 10. Комментарии-промпты (24.04.2026, 11:40)

Добавлены комментарии-промпты, использованные при генерации модулей `Parsec` и `Attoparsec`.

5.4 Вопросы защиты и их решения

5.4.1 Вопрос 1 (задание дано 24.04.2026)

Составить выражение с `<$>`, `<*>`, `<*>` типа `Either a (String, Float)`.
Решение:

```
1 (,) <$> (Right "abc") <*> (Right (+1.0) <*> (Right (4.0 :: Float)))
```

Листинг 3: `Either a (String, Float)` через аппликативный функтор

Первый `<*>` применяет функцию `(+1.0)` к значению `4.0`, второй — конструктор пары `(,)` к `"abc"` и полученному `5.0`. Результат: `Right ("abc 5.0")`.

5.4.2 Вопрос 2 (задание дано 08.05.2026)

Составить выражение с `<$>`, `<*>`, `<*>` типа `Either a (Int, String)`.
Решение:

```
1 (,) <$> (Right (1 :: Integer)) <*> (Right (++ "1") <*> (Right ("abc" ::  
  ↪ String)))
```

Листинг 4: `Either a (Int, String)` через аппликативный функтор

Первый `<*>` применяет функцию `(++ "1")` к строке `"abc"`, второй — конструктор пары к `1` и `"abc1"`. Результат: `Right (1, "abc1")`.

6 Лабораторная работа №3

Третья работа состоит из двух частей: генерация глитч-эффектов на изображениях и поиск пути в лабиринте с использованием монады RWS.

6.1 Часть 1. Глитч-эффекты

6.1.1 Коммит 1. Создание проектов (06.05.2026, 17:36)

Добавлены файлы stack-проектов `myGlithes` и `myMaze`.

6.1.2 Коммит 2. Функции глитч-эффектов (07.05.2026, 13:17)

В модуле `Glitches` реализованы:

- a) `intToChar` — преобразование `Int` в `Char` по модулю 255;
- b) `intToBC` — преобразование `Int` в `ByteString` из одного символа;
- c) `replaceByte` — детерминированная замена байта в заданной позиции;
- d) `randomReplaceByte` — замена случайного байта на случайное значение;
- e) `sortSection` — сортировка участка `ByteString` по заданному смещению и размеру;
- f) `randomSortSection` — сортировка случайного участка фиксированного размера (25 байт).

В `Main.hs` имя файла берётся из аргументов командной строки, к изображению последовательно применяется цепочка глитч-функций (через `foldM`), каждая версия сохраняется в отдельный файл.

6.2 Часть 2. Лабиринт на монаде RWS

6.2.1 Коммит 3. Типы данных лабиринта (07.05.2026, 18:50)

В модуле `Maze` определены типы: `RoomType = Start String | Finish String | Node String`, `Room = Room RoomType [String]` (тип комнаты и список соседей), `Maze = [Room]`. Реализованы функции `getName`, `getNeighbours`, `isFinish`, `findFinish`, `findStart`.

6.2.2 Коммит 4. Поиск пути (08.05.2026, 00:24)

Добавлены функции `findRoom`, `isInPath`, `neighboursCount`, `getUnvisited`, а также основной решатель `solveMaze` на монаде `RWS`: окружение — лабиринт, состояние — пройденный путь, лог — журнал перемещений. Реализован вспомогательный обход `searchPath` (поиск в глубину) с логированием и обработкой ошибок (отсутствие старта/финиша, тупик). По требованию задания до-нотация не используется.

6.2.3 Коммит 5. Тестирование и очистка (08.05.2026, 02:33)

В `Main.hs` добавлены тестовые лабиринты `schoolMaze` и `castleMaze` с выводом результата, пути и логов. Функции `findStart`/`findFinish` переработаны для устранения предупреждений компилятора; из исходников удалена кириллица.

6.3 Вопросы защиты и их решения

6.3.1 Вопрос 1 (задание дано 08.05.2026)

Составить выражение вида `(_ >= _ >= _)` с монадой `State`.

Решение:

```
1 f1 = state (\x -> ("x", "log"))
2 f2 = (\x -> state (\x -> ("x", "log")))
3 runState (f1 >=> f2 >=> f2) "abc"
4 -- ("x", "log")
```

Листинг 5: Цепочка `>=` для монады `State`

7 Лабораторная работа №4

В четвёртой работе реализован собственный трансформер монад `MyStateT` и текстовая игра «прохождение лабиринта» на стеке из трёх трансформеров.

7.1 Коммиты и выполняемая работа

7.1.1 Коммит 1. Трансформер `MyStateT` (14.05.2026, 22:35)

Определён тип `newtype MyStateT s m a = MyStateT { runMyStateT :: s -> m (a, s) }`. Реализована функция `myState` (поднятие чистой функции преобразования состояния) и представители классов:

а) `Functor` (при условии `Functor m`);

- b) `Applicative` (при условии `Monad m`);
- c) `Monad` (при условии `Monad m`);
- d) `MonadTrans` с методом `lift`.

7.1.2 Коммит 2. Загрузка лабиринта (14.05.2026, 23:39)

В модуль `Maze` перенесены типы и вспомогательные функции из третьей работы. Добавлены `parseRoom :: String -> Room` и `loadMaze :: FilePath -> IO Maze` для чтения карты лабиринта из текстового файла. Старт и финиш распознаются по именам «старт» и «финиш».

7.1.3 Коммит 3. Игровая логика (15.05.2026, 01:32)

Определён стек трансформеров `type Game a = ReaderT Maze (WriterT [String] (MyStateT String IO)) a`: `ReaderT` — доступ к лабиринту, `WriterT` — логирование, `MyStateT` — текущая комната, `IO` — ввод-вывод. Реализованы функции `logPrint`, `getCRoom`, `setCRoom`, `runGame` (с явной терминацией трансформеров), основной игровой цикл `gameProc` и точка входа `playMaze`. Игрок перемещается по комнатам, может завершить игру вводом «q»; перемещения логируются и выводятся в конце.

7.1.4 Коммит 4. Тестовый лабиринт (15.05.2026, 01:33)

Добавлен файл `mazes/uni.maze` с тестовой картой лабиринта.

7.1.5 Коммит 5. Комментарии к `runGame` (15.05.2026, 17:56)

Добавлены подробные комментарии к функции `runGame`, поясняющие пошаговое развёртывание (терминацию) стека трансформеров.

7.2 Вопросы защиты и ответы на них

7.2.1 Вопрос 1

Как работают типы в строке `runMyStateT (runWriterT (runReaderT game maze)) startRoom`?

Решение: применение функций снимает трансформеры по очереди:

- 1) `runReaderT game maze` даёт `WriterT [String] (MyStateT String IO) a`;
- 2) `runWriterT (...)` даёт `MyStateT String IO (a, [String])`;
- 3) `runMyStateT (...) startRoom` даёт `IO ((a, [String]), String)`.

7.2.2 Вопрос 2

Что такое терминатор?

Решение: терминатор — это самая нижняя монада в стеке трансформеров, то есть базовая монада, поверх которой надстраиваются все остальные слои. В данном проекте терминатором является `IO`:

7.2.3 Вопрос 3

Что такое монадный трансформер?

Решение: монадный трансформер — это тип вида `T m a`, параметризованный другой монадой `m`, который добавляет к `m` новый эффект, сохраняя возможность использовать эффекты `m`. Трансформер реализует класс `MonadTrans` с методом `lift :: m a -> T m a`, позволяющим «поднимать» вычисления внутренней монады в расширенный стек.

7.2.4 Вопрос 4

Где запускается монада `IO`?

Решение: монада `IO` запускается через оператор `(<-)` в до-нотации.

8 Лабораторная работа №5

В пятой работе реализованы вспомогательные функции и набор свойств для их тестирования с помощью библиотеки `QuickCheck`.

8.1 Коммиты и выполняемая работа

8.1.1 Коммит 1. Создание проекта (21.05.2026, 22:50)

Создан `stack`-проект `myFunctionsAndTests`, в `package.yaml` добавлена зависимость `QuickCheck` для тестов.

8.1.2 Коммит 2. Тестируемые функции (21.05.2026, 23:27)

В модуле `Lib` реализованы:

- a) `normMod :: Int -> Int -> Int` — приведение числа к остатку по модулю через повторное вычитание/сложение (вспомогательная функция);
- b) `addMod :: Int -> Int -> Int -> Int` — сложение по модулю;
- c) `splitTokens :: String -> [String]` — разбиение строки на чередующиеся блоки слов и пробелов;

- d) `clean :: [String] -> [String]` — удаление пустых токенов;
- e) `reverseWords :: String -> String` — изменение порядка слов на обратный (реализована как `concat . reverse . splitTokens`).

8.1.3 Коммит 3. Генераторы и свойства (22.05.2026, 01:24)

В `test/Spec.hs` реализованы генераторы `genMod`, `genWord`, `genSpaces`, `genPhrase`, `genText` и свойства:

- a) `prop_addMod_mod` — согласованность результата с `mod`: `addMod x y m `mod` m == (x + y) `mod` m`;
- b) `prop_addMod_neutral` — нейтральный элемент: `addMod x 0 m == x `mod` m`;
- c) `prop_addMod_comm` — коммутативность: `addMod x y m == addMod y x m`;
- d) `prop_rev_empty` — перевернут пустой строки даёт пустую строку;
- e) `prop_rev_single` — строка из одного слова не меняется;
- f) `prop_rev_order` — порядок слов меняется на обратный;
- g) `prop_rev_involution` — двойное применение возвращает исходную строку.

8.1.4 Коммит 4. `verbose` и `withMaxSuccess` (22.05.2026, 01:29)

В `main` тесты обернуты в `verbose` и `withMaxSuccess 5` для подробного вывода ограниченного числа проверок.

А Приложение А. Листинги кода